

OOPS

Thursday, 8 January 2026

9:13 PM

new keyword dynamically allocates memory to a variable and returns its reference variable to it.

```
Student student1 = new Student();
```

Student1 stores reference to new Student object created in heap memory.

student1 object reference is stored in stack memory.

```
Student student1 = new Student();
```

Student student1 declares a reference variable at compile time.

new Student() creates the object at runtime on the heap and assigns its reference to student1.

<https://realpython.com/python3-object-oriented-programming/#what-is-object-oriented-programming-in-python>

Object-oriented programming is an approach for modeling concrete, real-world things, like cars, as well as relations between things, like companies and employees or students and teachers. OOP models real-world entities as software objects that have some data associated with them and can perform certain operations.

1. Class:

A class is a blueprint for how to define something.

A class is a user-defined data type. It consists of data members and member functions, which can be accessed and used by creating an instance of that class. It represents the set of properties or methods that are common to all objects of one type.

2. Object:

An Object is an instance of a Class. An object has an identity, state, and behavior. Each object contains data and code to manipulate the data. Objects can interact without having to know details of each other's data or code, it is sufficient to know the type of message accepted and type of response returned by the objects.

While the class is the blueprint, an instance is an object that's built from a class and contains real data. An instance of the Dog class is not a blueprint anymore. It's an actual dog with a name, like Miles, who's four years old.

1. **Encapsulation** allows you to bundle data (attributes) and behaviors (methods) within a class to create a cohesive unit. By defining methods to control access to attributes and its modification, encapsulation helps maintain data integrity and promotes modular, secure code.
2. **Inheritance** enables the creation of hierarchical relationships between classes, allowing a subclass to inherit attributes and methods from a parent class. This promotes code reuse and reduces duplication.
3. **Abstraction** focuses on hiding implementation details and exposing only the essential functionality of an object. By enforcing a consistent interface, abstraction simplifies interactions with objects, allowing developers to focus on what an object does rather than how it achieves its functionality.
4. **Polymorphism** allows you to treat objects of different types as instances of the same base type, as long as they implement a common interface or behavior. Python's [duck typing](#) make it especially

suited for polymorphism, as it allows you to access attributes and methods on objects without needing to worry about their actual class.

Types of Inheritance

Simple Inheritance

When you have a class that inherits from a single parent class, then you're using **single-base inheritance** or just **simple inheritance**.

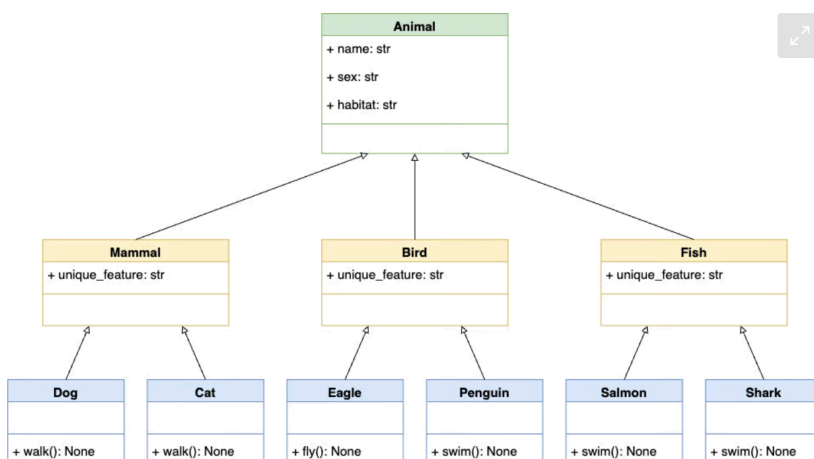
Class Hierarchies

Using inheritance, you can design and build [class hierarchies](#), also known as **inheritance trees**. A class hierarchy is a set of closely related classes that are connected through inheritance and arranged in a tree-like structure.

```
class Animal:
    def __init__(self, name, sex, habitat):
        self.name = name
        self.sex = sex
        self.habitat = habitat
```

```
class Mammal(Animal):
    unique_feature = "Mammary glands"
```

```
class Dog(Mammal):
    def walk(self):
        print("The dog is walking")
```



With class diagrams, you can also represent other types of relationships, including the following:

- **Composition** expresses a strong **has-a** relationship. For example, a robot has an arm. If the robot stops existing, then the arm stops existing too.
- **Aggregation** expresses a softer **has-a** relationship. For example, a university has an instructor. If the university stops existing, the instructor doesn't stop existing.

instructor. If the university stops existing, the instructor doesn't stop existing.

- **Association** expresses a **uses-a** relationship. For example, a student may be associated with a course. They will use the course. This relationship is common in database systems where you have one-to-one, one-to-many, and many-to-many associations.

Multiple Inheritance

In Python, you can use **multiple inheritance**. This type of inheritance allows you to create a class that inherits from several parent classes. The subclass will have access to attributes and methods from all its parents.

Benefits of Using Inheritance

Inheritance is a powerful tool that you can use to model and solve many real-world problems in your code. Some benefits of using inheritance include the following:

- **Reusability:** You can quickly inherit and reuse working code from one or more parent classes in as many subclasses as you need.
- **Modularity:** You can use inheritance to organize your code in hierarchies of related classes.
- **Maintainability:** You can quickly fix issues or add features to a parent class. These changes will be automatically available in all its subclasses. Inheritance also reduces code duplication.
- **Polymorphism:** You can create subclasses that can replace their parent class, providing the same or equivalent functionality.
- **Extensibility:** You can quickly extend an existing class by adding new data and behavior to its subclasses.

Disadvantages of Inheritance:

- Artificially increase your code's complexity with multiple inheritance or multiple levels of inheritance
- Face issues like the diamond problem where you'll have to deal with the method resolution order
- End up with [fragile base classes](#) where changes to a parent class produce unexpected behaviors in subclasses

Why do we need object-oriented programming

- To make the development and maintenance of projects more effortless.
- To provide the feature of data hiding that is good for security concerns.
- We can solve real-world problems if we are using object-oriented programming.
- It ensures code reusability.
- It lets us write generic code: which will work with a range of data, so we don't have to write basic stuff over and over again.

class Employee:

```
def __init__(self, name, age):  
    self.name = name  
    self.age = age
```

Problems with traditional approach:

```
kirk = ["James Kirk", 34, "Captain", 2265]  
spock = ["Spock", 35, "Science Officer", 2254]  
mccoy = ["Leonard McCoy", "Chief Medical Officer", 2266]
```

First, it can make larger code files more difficult to manage. If you reference `kirk[0]` several lines away from where you declared the `kirk` list, will you remember that the element with [index](#) 0 is the employee's name?

Second, it can introduce errors if employees don't have the same number of elements in their respective lists. In the mccoys list above, the age is missing, so mccoys[1] will return "Chief Medical Officer" instead of [Dr. McCoy's](#) age.

Attributes created in `__init__()` are called instance attribute.

[class attributes](#) are attributes that have the same value for all class instances.

After you create the Dog instances, you can access their instance attributes using **dot notation**:

```
>>> miles.name
'Miles'
>>> miles.age
4
```

You can access class attributes the same way

Instance methods are functions that you define inside a class and can only call on an instance of that class.

Python

```
>>> miles = Dog("Miles", 4)

>>> miles.description()
'Miles is 4 years old'

>>> miles.speak("Woof Woof")
'Miles says Woof Woof'

>>> miles.speak("Bow Wow")
'Miles says Bow Wow'
```

Python

dog.py

```
class Dog:
    # ...

    def __str__(self):
        return f"{self.name} is {self.age} years old"
```

Save the file and press `F5`. Now, when you print miles, you get a much

Python

```
>>> miles = Dog("Miles", 4)
>>> print(miles)
'Miles is 4 years old'
```

Methods like `__init__()` and `__str__()` are called **dunder methods** because they begin and end with [double](#)

[underscores.](#)

Inheritance

Python

```
class Parent:
    hair_color = "brown"

class Child(Parent):
    pass
```

Child classes inherit all of the parent's attributes and methods but can also specify attributes and methods that are unique to themselves.

Python

inheritance

```
class Parent:
    speaks = ["English"]

class Child(Parent):
    def __init__(self):
        super().__init__()
        self.speaks.append("German")
```

When you call `super().speak(sound)` inside `JackRussellTerrier`, Python searches the parent class, `Dog`, for a `.speak()` method and calls it with the variable `sound`.

```
# ...

class JackRussellTerrier(Dog):
    def speak(self, sound="Arf"):
        return super().speak(sound)

# ...
```

The `.__str__()` method provides what's known as the **informal string representation** of an object. This method must return a string that represents the object in a user-friendly manner.

The `.__repr__()` method is similar, but it must return a string that allows you to re-create the object if possible. So, this method returns what's known as the **formal string representation** of an object.

```

Python car.py

class Car:
    # ...

    def __str__(self):
        return f"{self.make} {self.model}, {self.color} ({self.year})"

    def __repr__(self):
        return (
            f"{type(self).__name__}"
            f' (make="{self.make}", '
            f'model="{self.model}", '
            f'year={self.year}, '
            f'color="{self.color}")'
        )

```

```

>>> from car import Car

>>> toyota_camry = Car("Toyota", "Camry", 2022, "Red")

>>> str(toyota_camry)
'Toyota Camry, Red (2022)'
>>> print(toyota_camry)
Toyota Camry, Red (2022)

>>> toyota_camry
Car(make="Toyota", model="Camry", year=2022, color="Red")
>>> repr(toyota_camry)
'Car(make="Toyota", model="Camry", year=2022, color="Red")'

```

Getter and Setter

Python

```

class Person:
    def __init__(self, name):
        self.set_name(name)

    def get_name(self):
        return self._name

    def set_name(self, value):
        self._name = value

```

Enumerations

Enums allow you to create sets of [named constants](#), which are known as **members** and can be accessed through the enumeration itself.

Python doesn't have a built-in enum data type. However, Python 3.4 introduced the [enum](#) module to provide the [Enum](#) class for supporting general-purpose enumerations.